Listen

Week 11 Lab: Introduction to OOP (Object-Oriented Programming)

What Are We Building?

Last week you wrote functions. This week you'll learn to organize code into **classes** — the building blocks of Object-Oriented Programming. You'll take things you already know (port scanning from Assignment 2, password checking, vulnerability reporting) and rebuild them as classes.

A class is like a blueprint. An object is a thing built from that blueprint. That's it.

Open the starter files and look for the `# TODO` markers.

Q1: Port Scanner Class

File: `lab11_q1_starter.py`

In Assignment 2, you wrote a port scanner using functions. Now you'll wrap that same logic into a **class**. The scanning code doesn't change — you're just organizing it differently. With a class, you can create multiple scanners, each with their own target and results, without any variables getting mixed up.

The starter already has the constructor and main section. **Your work is three methods** that do the actual scanning and display.

`scan_port(self, port)` — check if one port is open:

```bash
Create a socket (same as Assignment 2)
```

```
Set timeout to 1 second
Use connect_ex to check if the port is open
If open: print it, append to self.open_ports, return True
Otherwise: return False
Always close the socket (use try/finally)
```

`scan_range(self, start_port, end_port)` — scan a range of ports:

```bash
Loop from start_port to end_port (inclusive)
Call self.scan_port(port) for each one
```

`display_results(self)` — show what was found:

```bash
Print "Results for {self.target}:"
If self.open_ports is empty: print "No open ports found."
Otherwise: print each port number
```

Reference

```python
# This is what you already know (procedural):
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(1)
result = sock.connect_ex(("127.0.0.1", 80))
sock.close()


# The ONLY new thing is putting it inside a class:
class MyClass:
    def __init__(self, some_value):    # constructor — runs when you create
an object
        self.some_value = some_value   # self. stores data IN the object

    def my_method(self):               # a method — a function that belongs
to the class
        print(self.some_value)         # self. accesses data FROM the
object

# Creating an object:
obj = MyClass("hello")
obj.my_method()                        # prints "hello"
```

Test It

Run `python lab11_q1_starter.py`. You should see something like:

```bash
============================================================
  Q1: PORT SCANNER CLASS
============================================================

--- Scanner 1: localhost ---
  Scanning 127.0.0.1 ports 78-82...
  Port 80: OPEN
  Results for 127.0.0.1:
    Port 80

--- Scanner 2: different target ---
  Scanning 127.0.0.1 ports 20-25...
  Results for 127.0.0.1:
    No open ports found.

============================================================
```

Q2: Password Strength Checker Class

File: [lab11_q2_starter.py]

For the term project, your team will be looking for weak passwords and default credentials on `0x10.cloud`. But how do you know if a password is weak? You'll build a class that checks password strength.

The starter already has the main section and test passwords. **Your work is the four methods** inside the class.

`check_common(self, password)` — check if it's a known weak password:

```bash
Return True if password.lower() is in self.common_passwords
Return False otherwise
```

`check_strength(self, password)` — check three criteria:

```bash
has_length  = len(password) >= 8
has_digit   = any(c.isdigit() for c in password)
has_special = any(c in "!@#$%^&*" for c in password)
Return a dictionary: {"length": has_length, "digit": has_digit, "special":
```

```
    has_special}
```

`evaluate(self, password)` — combine both checks:

```bash
If check_common is True → "WEAK (common password)"
Otherwise, count how many strength checks pass:
  0 or 1 = "WEAK"
  2 = "MEDIUM"
  3 = "STRONG"
Append (password, result) to self.history
Return the result
```

Reference

```python
# any() checks if at least one item is True
any(c.isdigit() for c in "hello123")    # True
any(c.isdigit() for c in "hello")       # False

# Counting True values
checks = [True, False, True]
sum(checks)    # 2 — True counts as 1, False as 0

# Dictionary values
d = {"a": True, "b": False, "c": True}
sum(d.values())    # 2
```

Test It

Run `python lab11_q2_starter.py`. You should see:

```bash
============================================================
  Q2: PASSWORD STRENGTH CHECKER
============================================================

--- Checking Passwords ---
  admin           → WEAK (common password)
  hello           → WEAK
  hello123        → MEDIUM
  MyP@ss99        → STRONG
  p@ssw0rd!       → STRONG
  root            → WEAK (common password)

--- Check History ---
  admin           : WEAK (common password)
```

```
  hello           : WEAK
  hello123        : MEDIUM
  MyP@ss99        : STRONG
  p@ssw0rd!       : STRONG
  root            : WEAK (common password)

==============================================================
```

Q3: Vulnerability Report Class

File: [lab11_q3_starter.py]

For the term project, each team member finds a vulnerability and writes a report. You'll build a `Finding` class that represents a single vulnerability, and a `Report` class that collects multiple findings.

The starter already has the constructors, `__str__`, and main section. **Your work is three methods** in the `Report` class.

`add_finding(self, finding)` — add a finding to the report:

```bash
Append the finding to self.findings
```

`get_by_severity(self, severity)` — filter findings:

```bash
Use a list comprehension to return only findings where f.severity == severity
```

`summary(self)` — print a formatted report:

```bash
Print team name
Print total findings count
Print count per severity (HIGH, MEDIUM, LOW)
Print a separator line
Print each finding (print(f) calls __str__ automatically)
```

Reference

```python
```

```python
# __str__ controls what print() shows
class Dog:
    def __init__(self, name):
        self.name = name
    def __str__(self):
        return f"Dog named {self.name}"

d = Dog("Rex")
print(d)    # "Dog named Rex" — Python calls __str__ automatically

# List comprehension to filter objects
results = [f for f in my_list if f.severity == "HIGH"]
```

Test It

Run `python lab11_q3_starter.py`. You should see:

```bash
============================================================
  Q3: VULNERABILITY REPORT
============================================================

--- Adding Findings ---
  Added: [HIGH] ssh.0x10.cloud — Default credentials admin:admin
  Added: [LOW] blog.0x10.cloud — No HTTPS (cleartext)
  Added: [HIGH] ftp.0x10.cloud — Anonymous FTP access
```

Download Print Open with docReader

Activity Details

Task: View this topic